System Verification and Validation Plan for Flexin'

Team 3, Flexin'
Arjun Karthik
Shao (Daniel) Hu
Noah Goldschmied
Peter Deforest
Daniel Franze-Da Silva

October 29, 2026

Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

[The intention of the VnV plan is to increase confidence in the software. However, this does not mean listing every verification and validation technique that has ever been devised. The VnV plan should also be a **feasible** plan. Execution of the plan should be possible with the time and team available. If the full plan cannot be completed during the time available, it can either be modified to "fake it", or a better solution is to add a section describing what work has been completed and what work is still planned for the future. —SS]

[The VnV plan is typically started after the requirements stage, but before the design stage. This means that the sections related to unit testing cannot initially be completed. The sections will be filled in after the design stage is complete. the final version of the VnV plan should have all sections filled in. —SS]

Contents

1 Symbols, Abbreviations, and Acronyms **iv**

2 General Information **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 1
- 2.3 Custom Documents . . . 1
- 2.4 Relevant Documentation . . . 2

3 Plan **2**
- 3.1 Verification and Validation Team . . . 2
- 3.2 SRS Verification . . . 2
- 3.3 Design Verification . . . 2
- 3.4 Verification and Validation Plan Verification . . . 3
- 3.5 Implementation Verification . . . 3
- 3.6 Automated Testing and Verification Tools . . . 3
- 3.7 Software Validation . . . 4

4 System Tests **4**
- 4.1 Tests for Functional Requirements . . . 4
  - 4.1.1 Account Management . . . 4
  - 4.1.2 Fitness Tracking and Goals . . . 5
  - 4.1.3 Rewards and Virtual Currency . . . 5
  - 4.1.4 Simulated Investing . . . 5
  - 4.1.5 Community and Leaderboards . . . 6
  - 4.1.6 Progress Reporting . . . 6
- 4.2 Tests for Nonfunctional Requirements . . . 6
  - 4.2.1 Look and Feel . . . 6
  - 4.2.2 Usability and Humanity . . . 7
  - 4.2.3 Performance . . . 7
  - 4.2.4 Operational and Environmental . . . 8
  - 4.2.5 Maintainability and Support . . . 8
  - 4.2.6 Security . . . 8
  - 4.2.7 Cultural . . . 9
  - 4.2.8 Compliance . . . 9
  - 4.2.9 User Documentation and Training . . . 9
- 4.3 Traceability Between Test Cases and Requirements . . . 10

5 Unit Test Description **10**
5.1 Unit Testing Scope 10
5.2 Tests for Functional Requirements 10
5.2.1 Module 1 11
5.2.2 Module 2 11
5.3 Tests for Nonfunctional Requirements 12
5.3.1 Module ? 12
5.3.2 Module ? 12
5.4 Traceability Between Test Cases and Modules 12

6 Appendix **13**
6.1 Symbolic Parameters 13
6.2 Usability Survey Questions? 13

7 Appendix — Generative AI Use Disclosure **14**
7.1 AI Tools Used 14
7.2 How and Where Used 14
7.3 Verification of AI-Generated Content 14

8 Appendix — Reflection **15**

List of Tables

[Remove this section if it isn't needed —SS]

List of Figures

[Remove this section if it isn't needed —SS]

1 Symbols, Abbreviations, and Acronyms

symbol	description
T	Test

[symbols, abbreviations, or acronyms — you can simply reference the SRS (**?**) tables, if appropriate —SS]

[Remove this section if it isn't needed —SS]

This document ... [provide an introductory blurb and roadmap of the Verification and Validation plan —SS]

2 General Information

2.1 Summary

[Say what software is being tested. Give its name and a brief overview of its general functions. —SS]

2.2 Objectives

[State what is intended to be accomplished. The objective will be around the qualities that are most important for your project. You might have something like: "build confidence in the software correctness," "demonstrate adequate usability." etc. You won't list all of the qualities, just those that are most important. —SS]

[You should also list the objectives that are out of scope. You don't have the resources to do everything, so what will you be leaving out. For instance, if you are not going to verify the quality of usability, state this. It is also worthwhile to justify why the objectives are left out. —SS]

[The objectives are important because they highlight that you are aware of limitations in your resources for verification and validation. You can't do everything, so what are you going to prioritize? As an example, if your system depends on an external library, you can explicitly state that you will assume that external library has already been verified by its implementation team. —SS]

2.3 Custom Documents

[Summarize the CDs (Custom Documents) that were tackled by this project. CDs can include usability testing, code walkthroughs, user documentation, formal proof, GenderMag personas, Design Thinking, etc. CDs should have already been approved by the course instructor as included in your problem statement. You can use a pull request to update your CDs (in TeamComposition.csv or Repos.csv) if your plan changes as a result of the VnV planning exercise. —SS]

2.4 Relevant Documentation

[Reference relevant documentation. This will definitely include your SRS and your other project documents (design documents, like MG, MIS, etc). You can include these even before they are written, since by the time the project is done, they will be written. You can create BibTeX entries for your documents and within those entries include a hyperlink to the documents. —SS]

?

[Don't just list the other documents. You should explain why they are relevant and how they relate to your VnV efforts. —SS]

3 Plan

[Introduce this section. You can provide a roadmap of the sections to come. —SS]

3.1 Verification and Validation Team

[Your teammates. Maybe your supervisor. You should do more than list names. You should say what each person's role is for the project's verification. A table is a good way to summarize this information. —SS]

3.2 SRS Verification

[List any approaches you intend to use for SRS verification. This may include ad hoc feedback from reviewers, like your classmates (like your primary reviewer), or you may plan for something more rigorous/systematic. —SS]

[If you have a supervisor for the project, you shouldn't just say they will read over the SRS. You should explain your structured approach to the review. Will you have a meeting? What will you present? What questions will you ask? Will you give them instructions for a task-based inspection? Will you use your issue tracker? —SS]

[Maybe create an SRS checklist? —SS]

3.3 Design Verification

[Plans for design verification —SS]

[The review will include reviews by your classmates —SS]
[Create a checklists? —SS]

3.4 Verification and Validation Plan Verification

[The verification and validation plan is an artifact that should also be verified. Techniques for this include review and mutation testing. —SS]
[The review will include reviews by your classmates —SS]
[Create a checklists? —SS]

3.5 Implementation Verification

[You should at least point to the tests listed in this document and the unit testing plan. —SS]

[In this section you would also give any details of any plans for static verification of the implementation. Potential techniques include code walkthroughs, code inspection, static analyzers, etc. —SS]

[The final class presentation in CAS 741 could be used as a code walkthrough. There is also a possibility of using the final presentation (in CAS741) for a partial usability survey. —SS]

3.6 Automated Testing and Verification Tools

[What tools are you using for automated testing. Likely a unit testing framework and maybe a profiling tool, like ValGrind. Other possible tools include a static analyzer, make, continuous integration tools, test coverage tools, etc. Explain your plans for summarizing code coverage metrics. Linters are another important class of tools. For the programming language you select, you should look at the available linters. There may also be tools that verify that coding standards have been respected, like flake9 for Python. —SS]

[If you have already done this in the development plan, you can point to that document. —SS]

[The details of this section will likely evolve as you get closer to the implementation. —SS]

3.7 Software Validation

[If there is any external data that can be used for validation, you should point to it here. If there are no plans for validation, you should state that here. —SS]

[You might want to use review sessions with the stakeholder to check that the requirements document captures the right requirements. Maybe task based inspection? —SS]

[For those capstone teams with an external supervisor, the Rev 0 demo should be used as an opportunity to validate the requirements. You should plan on demonstrating your project to your supervisor shortly after the scheduled Rev 0 demo. The feedback from your supervisor will be very useful for improving your project. —SS]

[For teams without an external supervisor, user testing can serve the same purpose as a Rev 0 demo for the supervisor. —SS]

[This section might reference back to the SRS verification section. —SS]

4 System Tests

[There should be text between all headings, even if it is just a roadmap of the contents of the subsections. —SS]

4.1 Tests for Functional Requirements

The subsections below group the functional requirements in the Functional Requirements section of the SRS (**?**) by feature area. Together they cover FUNC-01 to FUNC-16. FUNC-17 (Sponsored Reward Redemption) is a stretch goal in the SRS Waiting Room, so it is not tested.

[Each test uses the fields Control, Initial State, Input, Output, Test Case Derivation, and How test will be performed. —SS]

4.1.1 Account Management

These tests cover creating an account and signing in.

SRS ID	Requirement
FUNC-01	User Registration
FUNC-02	User Authentication

[Add the test cases for these requirements. —SS]

4.1.2 Fitness Tracking and Goals

These tests cover importing activity data from third-party providers and creating and tracking fitness goals.

SRS ID	Requirement
FUNC-03	Third-Party Fitness Data Import
FUNC-04	Fitness Goal Creation
FUNC-05	Fitness Goal Tracking

[Add the test cases for these requirements. —SS]

4.1.3 Rewards and Virtual Currency

These tests cover awarding virtual currency for activity and keeping each user's balance.

SRS ID	Requirement
FUNC-06	Reward Allocation
FUNC-07	Virtual Currency Tracking

[Add the test cases for these requirements. —SS]

4.1.4 Simulated Investing

These tests cover the portfolio and trading features of the simulated market.

SRS ID	Requirement
FUNC-08	Portfolio Creation
FUNC-09	Asset Purchase
FUNC-10	Asset Sale
FUNC-11	Transaction Recording
FUNC-12	Portfolio Performance Calculation

[Add the test cases for these requirements. —SS]

4.1.5 Community and Leaderboards

These tests cover friends lists and public and private leaderboard groups.

SRS ID	Requirement
FUNC-13	Leaderboard Participation
FUNC-14	Friend Management
FUNC-15	Private Leaderboards

[Add the test cases for these requirements. —SS]

4.1.6 Progress Reporting

These tests cover the summaries of activity, goals, earnings, portfolio performance, and standings.

SRS ID	Requirement
FUNC-16	Progress Reporting

[Add the test cases for these requirements. —SS]

4.2 Tests for Nonfunctional Requirements

The subsections below follow the nonfunctional requirement sections of the SRS (**?**). Each one lists the SRS subsections and requirement IDs that its tests cover.

[Each test uses the fields Type, Initial State, Input/Condition, Output/Result, and How test will be performed. Static tests, such as reviews, inspections, and walkthroughs, should instead say who is involved and how the review is run. —SS]

4.2.1 Look and Feel

These tests cover the Look and Feel Requirements section of the SRS.

Appearance Covers LF-APP-01 to LF-APP-05.

[Add the test cases for these requirements. —SS]

Style Covers LF-STY-01 to LF-STY-04.

[Add the test cases for these requirements. —SS]

4.2.2 Usability and Humanity

These tests cover the Usability and Humanity Requirements section of the SRS.

Ease of Use Covers UHR-EOU-01 to UHR-EOU-09.
[Add the test cases for these requirements. —SS]

Personalization and Internationalization Covers UHR-PER-01 to UHR-PER-04, UHR-INT-01.
[Add the test cases for these requirements. —SS]

Learning Covers UHR-LRN-01 to UHR-LRN-04.
[Add the test cases for these requirements. —SS]

Understandability and Politeness Covers UHR-UND-01 to UHR-UND-03, UHR-POL-01 to UHR-POL-05.
[Add the test cases for these requirements. —SS]

Accessibility Covers UHR-ACC-01 to UHR-ACC-06.
[Add the test cases for these requirements. —SS]

4.2.3 Performance

These tests cover the Performance Requirements section of the SRS.

Speed and Latency Covers PR-SL-01 to PR-SL-04.
[Add the test cases for these requirements. —SS]

Safety-Critical Covers PR-SC-01 to PR-SC-08.
[Add the test cases for these requirements. —SS]

Precision or Accuracy Covers PR-PA-01 to PR-PA-06.
[Add the test cases for these requirements. —SS]

Robustness or Fault-Tolerance Covers PR-RFT-01 to PR-RFT-04.
[Add the test cases for these requirements. —SS]

Capacity Covers PR-C-01, PR-C-02.
[Add the test cases for these requirements. —SS]

Scalability or Extensibility Covers PR-SE-01 to PR-SE-03.
[Add the test cases for these requirements. —SS]

Longevity Covers PR-L-01, PR-L-02.
[Add the test cases for these requirements. —SS]

4.2.4 Operational and Environmental

These tests cover the Operational and Environmental Requirements section of the SRS.

Productization Covers PROD-01, PROD-02.
[Add the test cases for these requirements. —SS]

4.2.5 Maintainability and Support

These tests cover the Maintainability and Support Requirements section of the SRS.

Maintenance Covers MAS-MNT-01 to MAS-MNT-05.
[Add the test cases for these requirements. —SS]

Supportability Covers MAS-SUP-01.
[Add the test cases for these requirements. —SS]

Adaptability Covers MAS-ADPT-01 to MAS-ADPT-03.
[Add the test cases for these requirements. —SS]

4.2.6 Security

These tests cover the Security Requirements section of the SRS.

Access Covers SEC-ACC-01 to SEC-ACC-05.
[Add the test cases for these requirements. —SS]

Integrity Covers SEC-INT-01 to SEC-INT-03.
[Add the test cases for these requirements. —SS]

Privacy Covers SEC-PRV-01 to SEC-PRV-03.
[Add the test cases for these requirements. —SS]

Audit Covers SEC-AUD-01, SEC-AUD-02.
[Add the test cases for these requirements. —SS]

Immunity Covers SEC-IMM-01 to SEC-IMM-03.
[Add the test cases for these requirements. —SS]

4.2.7 Cultural

These tests cover the Cultural Requirements section of the SRS.

Cultural Covers CUL-01, CUL-02.
[Add the test cases for these requirements. —SS]

4.2.8 Compliance

These tests cover the Compliance Requirements section of the SRS.

Legal Covers COMP-LEG-01 to COMP-LEG-07.
[Add the test cases for these requirements. —SS]

Standards Compliance Covers COMP-STD-01 to COMP-STD-07.
[Add the test cases for these requirements. —SS]

4.2.9 User Documentation and Training

These tests cover the User Documentation and Training Requirements section of the SRS.

User Documentation Covers DOC-01 to DOC-04.
[Add the test cases for these requirements. —SS]

Training Covers TRN-01 to TRN-03.

[Add the test cases for these requirements. —SS]

4.3 Traceability Between Test Cases and Requirements

[Provide a table that shows which test cases are supporting which requirements. —SS]

5 Unit Test Description

[This section should not be filled in until after the MIS (detailed design document) has been completed. —SS]

[Reference your MIS (detailed design document) and explain your overall philosophy for test case selection. —SS]

[To save space and time, it may be an option to provide less detail in this section. For the unit tests you can potentially layout your testing strategy here. That is, you can explain how tests will be selected for each module. For instance, your test building approach could be test cases for each access program, including one test for normal behaviour and as many tests as needed for edge cases. Rather than create the details of the input and output here, you could point to the unit testing code. For this to work, you code needs to be well-documented, with meaningful names for all of the tests. —SS]

5.1 Unit Testing Scope

[What modules are outside of the scope. If there are modules that are developed by someone else, then you would say here if you aren't planning on verifying them. There may also be modules that are part of your software, but have a lower priority for verification than others. If this is the case, explain your rationale for the ranking of module importance. —SS]

5.2 Tests for Functional Requirements

[Most of the verification will be through automated unit testing. If appropriate specific modules can be verified by a non-testing based technique. That can also be documented in this section. —SS]

5.2.1 Module 1

[Include a blurb here to explain why the subsections below cover the module. References to the MIS would be good. You will want tests from a black box perspective and from a white box perspective. Explain to the reader how the tests were selected. —SS]

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

2. test-id2

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

3. ...

5.2.2 Module 2

...

5.3 Tests for Nonfunctional Requirements

[If there is a module that needs to be independently assessed for performance, those test cases can go here. In some projects, planning for nonfunctional tests of units will not be that relevant. —SS]

[These tests may involve collecting performance data from previously mentioned functional tests. —SS]

5.3.1 Module ?

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

5.3.2 Module ?

...

5.4 Traceability Between Test Cases and Modules

[Provide evidence that all modules have been considered. —SS]

6 Appendix

This is where you can place additional information.

6.1 Symbolic Parameters

The definition of the test cases will call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance.

6.2 Usability Survey Questions?

[This is a section that would be appropriate for some projects. —SS]

7 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

7.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

7.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

7.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

8 Appendix — Reflection

[This section is not required for CAS 741 —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?